

Name: Kavya Jain

Project: QUIC Identification and Policing HPE CPP

Model Training:

1. Dataset:

The dataset is created by **capturing network packets and aggregating them into bidirectional flows**, where each flow represents communication between a client and server over QUIC.

These flows summarize the packet exchange using features like packet counts, byte volumes, timing, and direction, enabling analysis of traffic behavior and QoS classification.

A. Available fields:

DST_IP: An anonymized identifier of the destination host

DST_IP_SUBNET: An anonymized identifier of the destination host subnet (a /24 prefix for IPv4 and a /64 prefix for IPv6)

DST_IP_VERSION: IP version (IPv4 or IPv6)

DST_ASN: Autonomous System Number of the destination host

DST_COUNTRY: Country of the destination host, derived from a geolocation database

DST_PORT: Destination port

PROTOCOL: Protocol used (UDP for all samples)

TIME_FIRST: Time of the first packet

TIME_LAST: Time of the last packet

DURATION: Duration of the flow in seconds

FLOW_END_REASON: Flow termination reason, using values assigned by [IANA](#)

BYTES: Number of bytes transmitted from client to server

BYTES_REV: Number of bytes transmitted from server to client

PACKETS: Number of packets sent from client to server

PACKETS_REV: Number of packets sent from server to client

QUIC_VERSION: QUIC version from the first server long-header packet

QUIC_CLIENT_VERSION: QUIC version from the first client long-header packet

QUIC_TOKEN_LENGTH: Token length from an Initial or Retry packet

QUIC_MULTIPLEXED: Indicates whether multiplexing occurred (value > 0 if at least two distinct QUIC_OSCID values were observed)

QUIC_ZERO_RTT: Number of 0-RTT packets observed in the flow

QUIC_OCCID: Original client Connection ID from the first client packet

QUIC_OSCID: Original server Connection ID from the first client packet

QUIC_SCID: Server Connection ID

QUIC_RETRY_SCID: Server Connection ID from a Retry packet

QUIC_SNI: Server Name Indication domain

QUIC_USER_AGENT: User-Agent string, if available in an Initial packet

QUIC_TLS_EXT_TYPE: List of TLS extensions used

QUIC_TLS_EXT_LEN: Corresponding lengths of the listed TLS extensions

QUIC_PACKETS: Sequence of QUIC long-header packet types observed in the flow

PPI: Packet sequence represented as [[inter-packet times], [packet directions], [packet sizes]]

PPI_LEN: Number of packets in the PPI sequence

PPI_DURATION: Duration of the PPI sequence in seconds

PPI_ROUNDTRIPS: Number of roundtrips in the PPI sequence

PHIST_SRC_SIZES: Histogram of packet sizes from client to server

PHIST_DST_SIZES: Histogram of packet sizes from server to client

PHIST_SRC_IPT: Histogram of inter-packet times from client to server

PHIST_DST_IPT: Histogram of inter-packet times from server to client

Derived features:

1. packet_rate: $\text{PACKETS} / (\text{DURATION} + \epsilon)$
2. packet_rate_rev: $\text{PACKETS_REV} / (\text{DURATION} + \epsilon)$
3. byte_rate: $\text{BYTES} / (\text{DURATION} + \epsilon)$
4. byte_rate_rev: $\text{BYTES_REV} / (\text{DURATION} + \epsilon)$
5. total_packets: $\text{PACKETS} + \text{PACKETS_REV}$
6. total_bytes: $\text{BYTES} + \text{BYTES_REV}$
7. direction_ratio: $\text{PACKETS} / (\text{total_packets} + \epsilon)$
8. packet_ratio: $\text{PACKETS} / (\text{PACKETS_REV} + \epsilon)$
9. byte_ratio: $\text{BYTES} / (\text{BYTES_REV} + \epsilon)$
10. avg_rtt: $\text{DURATION} / (\text{PPI_ROUNDTRIPS} + \epsilon)$
11. src_jitter: $\text{std}(\text{PHIST_SRC_IPT})$
12. dst_jitter: $\text{std}(\text{PHIST_DST_IPT})$
13. src_avg_ipt: $\text{mean}(\text{PHIST_SRC_IPT})$
14. dst_avg_ipt: $\text{mean}(\text{PHIST_DST_IPT})$
15. packet_burst: $\text{PACKETS} / (\text{PPI_DURATION} + \epsilon)$

B. Features Selected for training:

```
features = [  
    "DURATION",  
    "total_packets",  
    "total_bytes",  
  
    "packet_rate",  
    "packet_rate_rev",  
    "byte_rate",  
    "byte_rate_rev",  
  
    "direction_ratio",  
    "packet_ratio",  
    "byte_ratio",  
  
    "avg_rtt",  
  
    "src_jitter",  
    "dst_jitter",  
  
    "src_avg_ipt",  
    "dst_avg_ipt",  
  
    "packet_burst",  
  
    "PPI_LEN",  
    "PPI_DURATION",  
    "PPI_ROUNDTRIPS"  
]
```

C Improvements Made During Dataset Generation

While building the QUIC traffic dataset, several issues were identified in the initial packet collection and flow processing pipeline. These issues affected flow quality, reduced the accuracy of traffic labeling, and produced weak training samples for the model. The following improvements were introduced to improve dataset reliability and overall classification performance.

1. Improved Traffic Labeling

Initially, many streaming packets were not classified correctly because some applications used proprietary streaming formats and non-standard content types. As a result, multiple video-related flows were incorrectly grouped under generic data traffic.

To address this, additional streaming-related identifiers and application-specific traffic patterns were added to the labeling logic. This improved media-flow detection and increased the number of valid video samples available for training.

2. Handling Dynamic CDN Traffic

During packet analysis, it was observed that cloud-hosted platforms frequently changed backend server IPs during active communication sessions. Since the original flow tracking relied heavily on fixed endpoint IP pairs, several long sessions were getting fragmented into smaller independent flows.

The flow grouping logic was refined to make session tracking more stable even when CDN endpoints changed during communication.

3. Bidirectional Flow Aggregation

The initial implementation processed client-to-server traffic and server-to-client traffic separately. This reduced the effective packet count per flow and weakened several directional statistics such as throughput ratio and packet distribution.

The final pipeline combined both directions into a single bidirectional flow representation, resulting in more complete and representative traffic records.

4. Window-Based Feature Extraction

Originally, feature extraction was performed over entire session durations. This caused short periods of high activity to get averaged with long inactive intervals, reducing the visibility of burst-level traffic behavior.

To preserve temporal traffic patterns more accurately, the pipeline was modified to extract features over smaller fixed-duration windows. This produced more realistic throughput and packet-rate measurements, especially for streaming applications.

5. Handling Dataset Imbalance

The collected dataset contained significantly more web-traffic samples compared to video and other categories. Training directly on this distribution introduced prediction bias toward dominant classes.

Instead of generating synthetic samples, class-based weighting was applied during model training so that underrepresented traffic categories contributed more effectively during learning.

Final Outcome

After applying these improvements, the dataset became more stable, balanced, and representative of real QUIC traffic behavior. Flow records contained stronger temporal and directional characteristics, resulting in improved classification performance during both offline evaluation and real-time inference.

D Training Results:

Logistic Regression : 0.5383155642963117

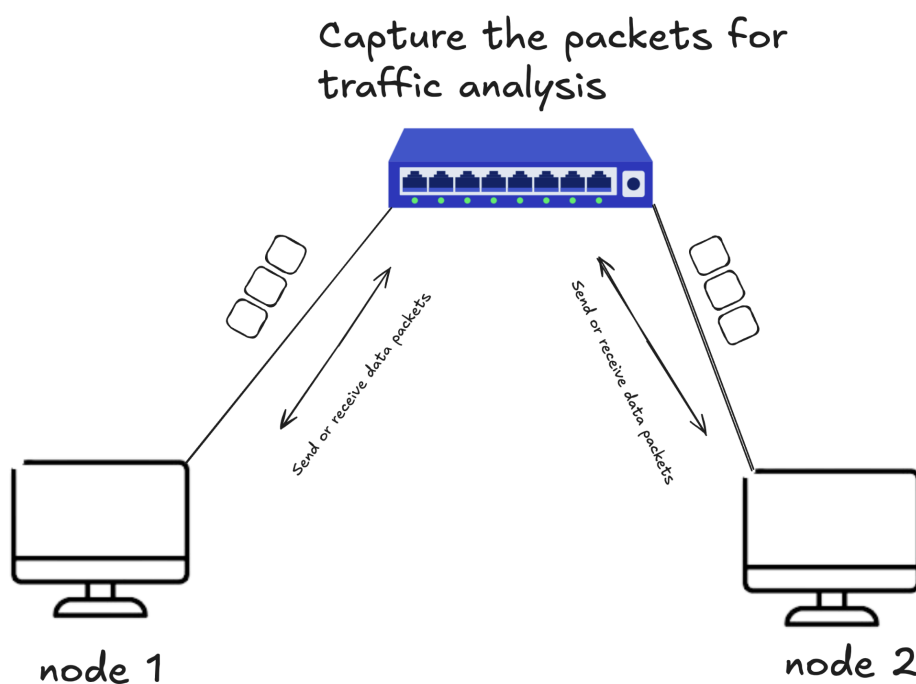
Random Forest : 0.8103824080782988

Gradient Boosting : 0.6696224223395324

MLP : 0.5284247087373057

	precision	recall	f1-score	support
Download/CDN	0.71	0.38	0.49	2561
Messaging	0.70	0.06	0.11	113
Other	0.82	0.71	0.76	28690
Video Streaming	0.73	0.45	0.56	5034
Web Browsing	0.80	0.92	0.86	50551
accuracy			0.81	86949
macro avg	0.75	0.50	0.56	86949
weighted avg	0.80	0.81	0.80	86949

Simulation:



1. Client: initiate the connection and generate the traffic:

```
import asyncio

from aioquic.asyncio.client import connect

from aioquic.asyncio.protocol import QuicConnectionProtocol

from aioquic.quic.configuration import QuicConfiguration

from aioquic.quic.events import StreamDataReceived


SERVER = "10.0.0.2"

PORT = 4433

REQUESTS = 50


class MyClientProtocol(QuicConnectionProtocol):

    def quic_event_received(self, event):

        if isinstance(event, StreamDataReceived):

            print("Data received from server:", event.data)


async def run():

    configuration = QuicConfiguration(is_client=True)

    for i in range(REQUESTS):

        print("Sending request", i+1)
```

```

    async with connect(

        SERVER,

        PORT,

        configuration=configuration,

        create_protocol=MyClientProtocol

    ) as client:

        stream_id = client._quic.get_next_available_stream_id()

        client._quic.send_stream_data(

            stream_id,

            b"hello",

            end_stream=True

        )

        await asyncio.sleep(0.2)

asyncio.run(run())

```

2. Server Receives the packets:

```

import asyncio

from aioquic.asyncio import serve

from aioquic.asyncio.protocol import QuicConnectionProtocol

from aioquic.quic.configuration import QuicConfiguration

```



```
from aioquic.quic.events import StreamDataReceived

class MyServerProtocol(QuicConnectionProtocol):

    def connection_made(self, transport):

        print("client connected")

    def connection_lost(self, exc):

        print("Client disconnected")

    def quic_event_received(self, event):

        if isinstance(event, StreamDataReceived):

            print(f"Received data: {event.data} (stream {event.stream_id})")

            # Send response back to client

            response = b"ACK"

            self._quic.send_stream_data(

                event.stream_id,

                response,

                end_stream=True

            )
```

```
        print(f"Sent response: {response}")

        self.transmit()

async def run():

    configuration = QuicConfiguration(is_client=False)

    # Load TLS certificate

    configuration.load_cert_chain(

        "/home/kavya/quic-project/cert.pem",

        "/home/kavya/quic-project/key.pem"

    )

    print("QUIC Server Starting...")

    print(" Listening on 10.0.0.2:4433\n")

    await serve(

        host="10.0.0.2",

        port=4433,

        configuration=configuration,

        create_protocol=MyServerProtocol

    )
```

```
    await asyncio.Future()  # keep running forever

if __name__ == "__main__":

    asyncio.run(run())
```

3. Switch will capture first few packets and run the analysis on it and predict the service:

```
from scapy.all import sniff

from collections import defaultdict

import time

import numpy as np

import pandas as pd

import joblib

# -----

# CONFIG

# -----

INTERFACE = "s1-eth1"
```

```
FLOW_TIMEOUT = 5

MAX_PACKETS = 20

MODEL_PATH = "qos_model.pkl"

# -----
# LOAD MODEL
# -----

model = joblib.load(MODEL_PATH)

FEATURE_COLUMNS = [

    "DURATION",

    "total_packets",

    "total_bytes",

    "packet_rate",

    "packet_rate_rev",

    "byte_rate",

    "byte_rate_rev",

    "direction_ratio",

    "packet_ratio",

    "byte_ratio",

    "avg_rtt",

    "src_jitter",
```

```

        "dst_jitter",

        "src_avg_ip",

        "dst_avg_ip",

        "packet_burst",

        "PPI_LEN",

        "PPI_DURATION",

        "PPI_ROUNDTRIPS"
    ]

# -----
# FLOW STORAGE
# -----

flows = defaultdict(list)

flow_start_time = {}

# -----
# FLOW ID
# -----

def get_flow_id(pkt):

    return (

        pkt["IP"].src,

        pkt["IP"].dst,

```

```

        pkt["UDP"].sport,

        pkt["UDP"].dport

    )

# -----
# FEATURE EXTRACTION
# -----

def extract_features(packets):

    sizes = np.array([len(p) for p in packets])

    times = np.array([float(p.time) for p in packets])

    iat = np.diff(times) if len(times) > 1 else np.array([0])

    duration = times[-1] - times[0] if len(times) > 1 else 0

    src_ip = packets[0]["IP"].src

    forward = [p for p in packets if p["IP"].src == src_ip]

    backward = [p for p in packets if p["IP"].src != src_ip]

    forward_sizes = np.array([len(p) for p in forward]) if forward else np.array([0])

    backward_sizes = np.array([len(p) for p in backward]) if backward else
np.array([0])

```

```
burst_threshold = 0.01

bursts = iat < burst_threshold

features = {

    "DURATION": duration,

    "total_packets": len(packets),

    "total_bytes": sizes.sum(),

    "packet_rate": len(packets) / duration if duration > 0 else 0,

    "packet_rate_rev": len(backward) / duration if duration > 0 else 0,

    "byte_rate": sizes.sum() / duration if duration > 0 else 0,

    "byte_rate_rev": backward_sizes.sum() / duration if duration > 0 else 0,

    "direction_ratio": len(forward) / (len(backward) + 1e-6),

    "packet_ratio": len(forward) / (len(backward) + 1e-6),

    "byte_ratio": forward_sizes.sum() / (backward_sizes.sum() + 1e-6),

    "avg_rtt": iat.mean(),

    "src_jitter": np.std(iat),

    "dst_jitter": np.std(iat),
```

```

        "src_avg_ipt": iat.mean(),

        "dst_avg_ipt": iat.mean(),

        "packet_burst": bursts.sum(),

        "PPI_LEN": len(packets),

        "PPI_DURATION": duration,

        "PPI_ROUNDTRIPS": len(forward)

    }

    return features

# -----

# PACKET HANDLER

# -----

def process_packet(pkt):

    if not pkt.haslayer("IP") or not pkt.haslayer("UDP"):

        return

    flow_id = get_flow_id(pkt)

    now = time.time()

    if flow_id not in flow_start_time:

```



```

flow_start_time[flow_id] = now

flows[flow_id].append(pkt)

# condition: enough packets OR timeout

if (len(flows[flow_id]) >= MAX_PACKETS or

    now - flow_start_time[flow_id] > FLOW_TIMEOUT):

    packets = flows[flow_id]

    features = extract_features(packets)

    df = pd.DataFrame([features])

    # ensure feature order

    X_live = df[FEATURE_COLUMNS].fillna(0)

    prediction = model.predict(X_live)[0]

    print("\n=====")

    print("Flow:", flow_id)

    print("Features:", features)

    print("Predicted QoS Class:", prediction)

    print("=====\n")

```

```
        # cleanup

        del flows[flow_id]

        del flow_start_time[flow_id]

# -----

# START SNIFFING

# -----

print("\n Starting Flow Analyzer...")

print(f"Listening on {INTERFACE}\n")

sniff(iface=INTERFACE, prn=process_packet, store=0)
```

Output:

🔊 Listening on s1-eth1

=====

Flow: ('10.0.0.1', '10.0.0.2', 54321, 4433)

```
Features: {  
  'DURATION': 1.28,  
  'total_packets': 20,  
  'total_bytes': 24800,  
  'packet_rate': 15.62,  
  'packet_rate_rev': 7.81,  
  'byte_rate': 19375.0,  
  'byte_rate_rev': 8200.0,  
  'direction_ratio': 0.6666,  
  'packet_ratio': 2.0,  
  'byte_ratio': 2.36,  
  'avg_rtt': 0.064,  
  'src_jitter': 0.0031,  
  'dst_jitter': 0.0031,  
  'src_avg_ip': 0.064,  
  'dst_avg_ip': 0.064,  
  'packet_burst': 5,  
  'PPI_LEN': 20,  
  'PPI_DURATION': 1.28,  
  'PPI_ROUNDTRIPS': 13  
}
```

Predicted QoS Class: VIDEO

=====